

Lección 4 — Development Environment for Advanced Techniques

1. Introducción General de la Lección

Esta lección marca la transición desde técnicas básicas de ciberseguridad hacia ambientes profesionales y reproducibles. Hasta este punto, las técnicas se ejecutaban directamente desde el sistema operativo principal utilizando Python e IDEs simples. Sin embargo, a medida que las técnicas avanzan, aparece la necesidad crítica de:

- **Aislar dependencias**: Evitar conflictos entre librerías
- **Usar múltiples versiones**: Trabajar con versiones específicas de librerías
- **Trabajar con distintos SOs**: Practicar en múltiples plataformas
- **Desplegar ambientes vulnerables**: Crear laboratorios de práctica
- **Construir laboratorios controlados**: Entornos seguros y reproducibles

La lección enseña cómo crear laboratorios reproducibles y seguros utilizando Python Virtual Environment (venv), virtualización con Hyper-V y VirtualBox, máquinas virtuales y distribuciones especializadas como Kali Linux.

2. Objetivos

Los objetivos de aprendizaje para esta lección son:

- Comprender fundamentos de virtualización y arquitectura de hypervisors.
- Crear laboratorios aislados de ciberseguridad reproducibles.
- Construir ambientes de pruebas seguros y controlados.
- Trabajar con Python Virtual Environment para aislar dependencias.
- Configurar máquinas virtuales usando Hyper-V y VirtualBox.
- Crear entornos seguros para offensive y defensive security.
- Comprender cómo construir superficies de pruebas locales.
- Validar conectividad y configuración de laboratorios.

3. Conceptos Técnicos Fundamentales

Virtualización

La virtualización consiste en crear versiones virtuales de hardware físico. Los componentes típicamente virtualizados incluyen CPU, memoria RAM, almacenamiento y networking. Estas instancias virtuales se denominan **Virtual Machines (VMs)**.

Una VM permite ejecutar un sistema operativo completo dentro de otro sistema operativo o sobre un hypervisor dedicado. Las ventajas incluyen aislamiento, snapshots, rollback, testing seguro, ambientes reproducibles, laboratorios de malware, análisis forense y pentesting controlado.

Hypervisor

El Hypervisor es el software encargado de crear, administrar, iniciar/detener VMs, manejar snapshots y asignar recursos. Existen dos grandes categorías:

****Bare Metal Hypervisor****: Corre directamente sobre hardware físico (VMware ESXi, Hyper-V Server).

****Hosted Hypervisor****: Corre sobre otro sistema operativo (VirtualBox, Hyper-V sobre Windows Desktop, VMware Workstation).

Host Machine vs Guest Machine

El ****host machine**** es el computador físico que controla recursos reales. El ****host operating system**** es el SO del host. El ****guest machine**** es la VM virtual. El ****guest operating system**** es el SO dentro de la VM.

Snapshots

Los snapshots permiten guardar el estado completo de una VM en un momento específico. Esto permite volver atrás, revertir errores, analizar malware de manera segura y restaurar entornos vulnerados rápidamente.

Attack Surface

Attack Surface representa el nivel de exposición de un sistema. Mientras más superficie expuesta exista, mayor probabilidad de ataque, mayor riesgo y más vectores disponibles. La superficie de ataque incluye puertos abiertos, servicios vulnerables, APIs, aplicaciones web, credenciales, protocolos inseguros y sistemas expuestos.

Target System

Un Target System es el sistema objetivo sobre el cual se realizan análisis, escaneos, pruebas y ataques controlados. Puede ser servidor físico, VM, base de datos, dispositivo IoT, impresora o endpoint. La lección recomienda utilizar únicamente sistemas propios, laboratorios controlados y ambientes autorizados.

Vulnerable / Exploitable VM

Las VMs vulnerables son fundamentales para aprender ciberseguridad. Permiten practicar explotación, probar scanners, validar técnicas, aprender offensive security y automatizar ataques controlados. Estas VMs simulan vulnerabilidades reales, servicios inseguros, configuraciones débiles y sistemas legacy.

Kali Linux

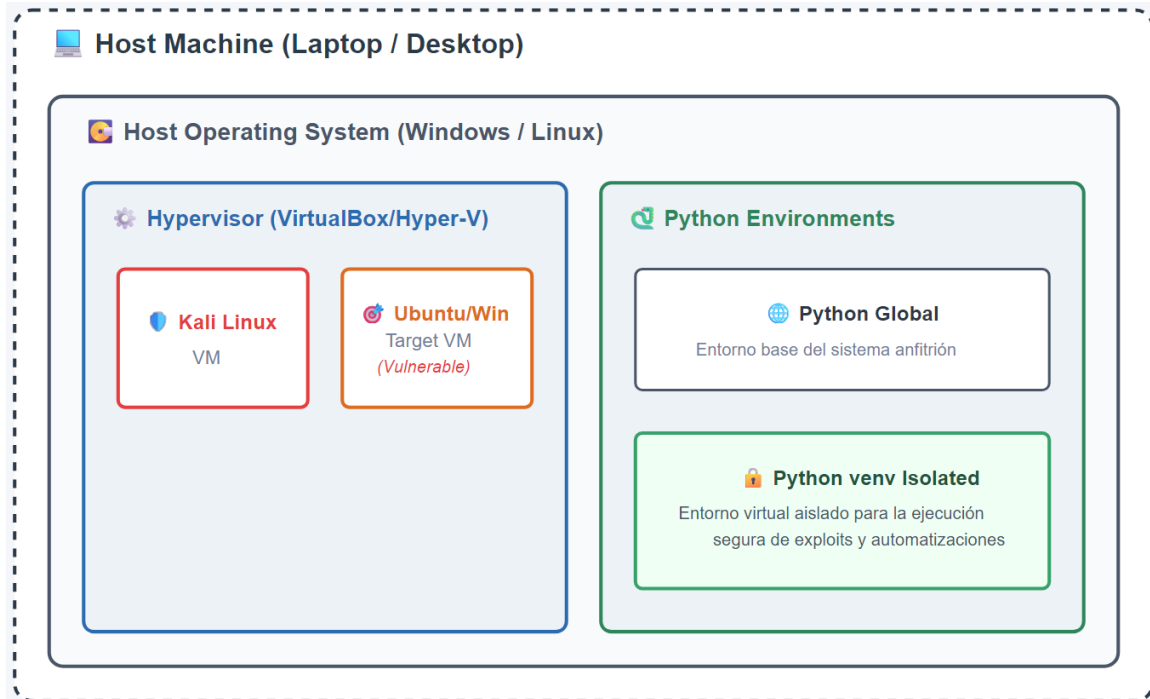
Kali Linux es una distribución Linux especializada en ciberseguridad basada en Debian. Incluye herramientas para penetration testing, digital forensics, network analysis, vulnerability assessment, exploitation y wireless testing. Mientras más avance el estudiante, más valor encontrará en Kali Linux.

Python Virtual Environment (venv)

Python venv permite crear ambientes Python aislados. Resuelve problemas de conflictos de dependencias, múltiples versiones de librerías, proyectos legacy, ambientes paralelos y testing seguro. Cada venv contiene binarios Python propios, librerías propias y dependencias propias.

4. Arquitectura o Flujo Técnico

El flujo típico de un laboratorio de ciberseguridad moderno sigue esta arquitectura:



5. Herramientas Utilizadas

Herramientas de Virtualización

- **VirtualBox**: Solución gratuita de Oracle, multiplataforma
- **Hyper-V**: Plataforma de virtualización de Microsoft, integrada en Windows
- **VMware**: Solución comercial profesional

Herramientas de Laboratorio

- **Kali Linux**: Distribución especializada en ciberseguridad
- **Ubuntu**: Sistema operativo base para laboratorios
- **Docker/Podman**: Containerización moderna

Librerías Python Requeridas

```
# Librerías estándar (incluidas en Python)
# - sys
# - subprocess
# - platform
```

```
# Librerías externas
pip install psutil
pip install icmplib
```

6. Scripts de la Lección

6.1 Script: `python_environment_info.py`

Introducción del Script

Este script verifica si Python se está ejecutando dentro de un virtual environment. Es una herramienta de diagnóstico fundamental para validar que el entorno aislado está correctamente activado.

****Propósito**:** Detectar si se está dentro de un venv y mostrar instrucciones de activación.

****Tipo**:** Herramienta de diagnóstico, validación de entorno.

Requisitos

- ****Librerías Python**:** `sys` (librería estándar)
- ****Sistemas operativos compatibles**:** Windows, Linux, macOS
- ****Requisitos de privilegios**:** Ninguno

Dependencias Externas

- Ninguna

Explicación Línea por Línea

```
import sys
```

Importa módulo `sys` que proporciona acceso a variables del intérprete.

```
print("=== Virtual Environment Check ===")
```

Imprime encabezado.

```
if sys.prefix != sys.base_prefix:
    print("Estás ejecutando Python dentro de un virtual
environment.")
    print(f"Virtual env path : {sys.prefix}")
```

Compara `sys.prefix` (ruta del Python actual) con `sys.base\_prefix` (ruta del Python del sistema). Si son diferentes, está dentro de venv.

```
else:
    print("NO estás dentro de un virtual environment.")
    print("Crea uno con:")
    print("python -m venv venv")
    print("Actívalo en Linux/macOS con:")
    print("source venv/bin/activate")
```

```
print("Actívalo en Windows con:")
print(r"venv\Scripts\activate.bat")
```

Si no está en venv, imprime instrucciones de creación y activación.

Cómo Ejecutar el Script

```
# Sin venv
python3 python_environment_info.py

# Con venv activado
source venv/bin/activate # Linux/macOS
python3 python_environment_info.py
```

Resultado Esperado

****Sin venv:****

```
=== Virtual Environment Check ===
NO estás dentro de un virtual environment.
Crea uno con:
python -m venv venv
Actívalo en Linux/macOS con:
source venv/bin/activate
Actívalo en Windows con:
venv\Scripts\activate.bat
```

****Con venv:****

```
=== Virtual Environment Check ===
Estás ejecutando Python dentro de un virtual environment.
Virtual env path : /home/user/project/venv
```

Riesgos y Consideraciones Éticas

- No hay riesgos de seguridad.
- Herramienta de diagnóstico únicamente.

Troubleshooting

- ****Salida incorrecta****: Verificar que el venv esté correctamente activado.

6.2 Script: network_interfaces.py

Introducción del Script

Este script enumera todas las interfaces de red del sistema y muestra información detallada como direcciones IP, máscaras de red, broadcast y familia de direcciones (IPv4/IPv6).

****Propósito****: Diagnosticar configuración de red del laboratorio.

****Tipo****: Herramienta de diagnóstico, validación de red.

Requisitos

- ****Librerías Python****: `psutil`
- ****Instalación****: `pip install psutil`
- ****Sistemas operativos compatibles****: Windows, Linux, macOS
- ****Requisitos de privilegios****: Ninguno

Dependencias Externas

- Ninguna

Explicación Línea por Línea

```
import psutil
```

Importa librería `psutil` que proporciona acceso a información del sistema.

```
print("=== Network Interfaces ===")
interfaces = psutil.net_if_addrs()
```

Obtiene diccionario de interfaces de red.

```
for interface_name, addresses in interfaces.items():
    print(f"\nInterface: {interface_name}")
```

Itera sobre cada interfaz de red.

```
for addr in addresses:
    print(f"  Family   : {addr.family}")
    print(f"  Address  : {addr.address}")
    print(f"  Netmask   : {addr.netmask}")
    print(f"  Broadcast : {addr.broadcast}")
```

Para cada dirección en la interfaz, imprime familia (IPv4/IPv6), dirección IP, máscara de red y broadcast.

Cómo Ejecutar el Script

```
python3 network_interfaces.py
```

Resultado Esperado

```
=== Network Interfaces ===

Interface: eth0
  Family   : AddressFamily.AF_INET
  Address  : 192.168.1.100
  Netmask   : 255.255.255.0
  Broadcast : 192.168.1.255

Interface: lo
  Family   : AddressFamily.AF_INET
  Address  : 127.0.0.1
```

```
Netmask : 255.255.255.0  
Broadcast: None
```

Riesgos y Consideraciones Éticas

- No hay riesgos de seguridad.
- Herramienta de diagnóstico únicamente.

Troubleshooting

- ****Error "ModuleNotFoundError"****: Ejecutar ``pip install psutil``.

6.3 Script: `vm_connectivity_test.py`

Introducción del Script

Este script prueba conectividad a una máquina virtual local ejecutando el comando ``ping``. Funciona de manera multiplataforma, adaptando el comando según Windows, Linux o macOS.

****Propósito****: Validar que una VM es accesible desde el host.

****Tipo****: Herramienta de diagnóstico, validación de conectividad.

Requisitos

- ****Librerías Python****: ``subprocess``, ``sys``, ``platform`` (librerías estándar)
- ****Sistemas operativos compatibles****: Windows, Linux, macOS
- ****Requisitos de privilegios****: Ninguno

Dependencias Externas

- Comando ``ping`` disponible en el SO.
- Acceso a red local.

Explicación Línea por Línea

```
import subprocess  
import sys  
import platform
```

Importa librerías para ejecutar comandos del SO, acceso a argumentos y detección del SO.

```
if len(sys.argv) != 2:  
    print("Uso: python 04_vm_connectivity_test.py <IP_VM>")  
    sys.exit(1)
```

Valida que se proporcionó exactamente un argumento (IP de la VM).

```
target = sys.argv[1]  
print(f"=== Connectivity Test to VM: {target} ===")
```

Obtiene IP objetivo de argumentos de línea de comandos.

```
system = platform.system().lower()

if system == "windows":
    command = ["ping", "-n", "4", target]
else:
    command = ["ping", "-c", "4", target]
```

Detecta el SO y construye comando `ping` apropiado:

- ****Windows****: `ping -n 4 target` (4 paquetes)
- ****Linux/macOS****: `ping -c 4 target` (4 paquetes)

```
try:
    result = subprocess.run(command, capture_output=True,
text=True)
    print(result.stdout)
    if result.returncode == 0:
        print("Resultado: VM alcanzable.")
    else:
        print("Resultado: VM no alcanzable.")
except Exception as error:
    print(f"Error ejecutando ping: {error}")
```

Ejecuta comando, captura salida y verifica código de retorno.

Cómo Ejecutar el Script

```
python3 vm_connectivity_test.py 192.168.56.101
```

Resultado Esperado

```
=== Connectivity Test to VM: 192.168.56.101 ===
PING 192.168.56.101 (192.168.56.101) 56(84) bytes of data.
64 bytes from 192.168.56.101: icmp_seq=1 ttl=64 time=0.5 ms
64 bytes from 192.168.56.101: icmp_seq=2 ttl=64 time=0.4 ms
64 bytes from 192.168.56.101: icmp_seq=3 ttl=64 time=0.5 ms
64 bytes from 192.168.56.101: icmp_seq=4 ttl=64 time=0.4 ms

--- 192.168.56.101 statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3ms
rtt min/avg/max/stddev = 0.4/0.45/0.5/0.04 ms

Resultado: VM alcanzable.
```

Riesgos y Consideraciones Éticas

- No hay riesgos de seguridad.
- Herramienta de diagnóstico únicamente.

Troubleshooting

- ****"VM no alcanzable"****: Verificar que la VM esté encendida y la IP sea correcta.
- ****"Comando no encontrado"****: Verificar que `ping` esté disponible en el SO.

6.4 Script: traceroute_local_vm1.py

Introducción del Script

Este script ejecuta traceroute hacia una máquina virtual local utilizando comandos del sistema operativo. Funciona de manera multiplataforma, usando `traceroute` en Linux/macOS y `tracert` en Windows.

****Propósito**:** Mapear ruta de red hacia una VM local.

****Tipo**:** Herramienta de diagnóstico, análisis de rutas.

Requisitos

- ****Librerías Python**:** `subprocess`, `sys`, `platform` (librerías estándar)
- ****Herramientas del SO**:** `traceroute` (Linux/macOS) o `tracert` (Windows)
- ****Sistemas operativos compatibles**:** Windows, Linux, macOS
- ****Requisitos de privilegios**:** Ninguno

Dependencias Externas

- Comando `traceroute` o `tracert` disponible en el SO.
- Acceso a red local.

Explicación Línea por Línea

```
import subprocess
import sys
import platform
```

Importa librerías necesarias.

```
if len(sys.argv) != 2:
    print("Uso: python 05_traceroute_local_vm.py <IP_VM>")
    sys.exit(1)
```

Valida que se proporcionó exactamente un argumento.

```
target = sys.argv[1]
print(f"=== Traceroute to VM: {target} ===")
```

Obtiene IP objetivo.

```
system = platform.system().lower()

if system == "windows":
    command = ["tracert", target]
else:
    command = ["traceroute", target]
```

Detecta SO y construye comando apropiado.

```
try:
    result = subprocess.run(command, capture_output=True,
text=True)
    print(result.stdout)
    if result.stderr:
        print("Errores:")
        print(result.stderr)
except Exception as error:
    print(f"Error ejecutando traceroute/tracert: {error}")
```

Ejecuta comando, captura salida y errores.

Cómo Ejecutar el Script

```
python3 traceroute_local_vm1.py 192.168.56.101
```

Resultado Esperado

```
=== Traceroute to VM: 192.168.56.101 ===
traceroute to 192.168.56.101 (192.168.56.101), 30 hops max, 60
byte packets
 1  192.168.56.101 (192.168.56.101)  0.5 ms  0.4 ms  0.5 ms
```

Las VMs locales típicamente aparecen a un solo hop, confirmando conectividad correcta.

Riesgos y Consideraciones Éticas

- No hay riesgos de seguridad.
- Herramienta de diagnóstico únicamente.

Troubleshooting

- ****"command not found"**: Instalar traceroute: `sudo apt install traceroute`.**
- ****Timeouts****: VM puede estar bloqueando ICMP.**

6.5 Script: traceroute_local_vm.py

Introducción del Script

Este script implementa traceroute en Python puro utilizando la librería ``icmplib``. Proporciona más control sobre parámetros y formatea la salida de manera estructurada.

****Propósito****: Mapear ruta de red hacia VMs locales con control fino.

****Tipo****: Herramienta de diagnóstico, análisis de rutas.

Requisitos

- ****Librerías Python****: ``icmplib``
- ****Instalación****: ``pip install icmplib``
- ****Sistemas operativos compatibles****: Windows, Linux, macOS
- ****Requisitos de privilegios****: Privilegios elevados (enviar ICMP)

Dependencias Externas

- Acceso a red local.
- Privilegios administrativos.

Explicación Línea por Línea

```
from icmplib import traceroute
```

Importa función `traceroute` de `icmplib`.

```
class TracerouteLocalVM:
    def check_traceroute(self, host) -> None:
        print('Checking hops for the host...')
        hops = traceroute(host, max_hops=20)
```

Define clase que ejecuta traceroute con máximo 20 hops.

```
        print('Distance/TTL \tAddress \tAverage round-trip time
\tPackets Sent')
        last_distance = 0
        for hop in hops:
            if last_distance + 1 != hop.distance:
                print('No response from gateway')
```

Itera sobre hops e imprime información formateada. Detecta gateways sin respuesta.

```
                print(f'{hop.distance:<15} {hop.address:<15}
{hop.avg_rtt} ms \t\t\t{hop.packets_sent:<5}')
                last_distance = hop.distance
```

Imprime distancia (TTL), dirección IP, latencia promedio y paquetes enviados.

```
if __name__ == "__main__":
    traceroute_check = TracerouteLocalVM()
    traceroute_check.check_traceroute('172.25.120.171')
    traceroute_check.check_traceroute('172.25.115.189')
```

Bloque de ejecución que traza rutas a dos VMs locales.

Cómo Ejecutar el Script

```
# Requiere privilegios elevados
sudo python3 traceroute_local_vm.py
```

Resultado Esperado

```
Checking hops for the host...
All Hops:
<Hops object>
```

Distance/TTL	Address	Average round-trip time	Packets Sent
1	172.25.120.171	0.5 ms	1

VMs locales típicamente aparecen a un solo hop.

Riesgos y Consideraciones Éticas

- No hay riesgos de seguridad.
- Herramienta de diagnóstico únicamente.

Troubleshooting

- ****Error "PermissionError"**: Ejecutar con `sudo`.**
- ****Error "ModuleNotFoundError"**: Ejecutar `pip install icmplib`.**

7. Construcción de un Cybersecurity Lab

Objetivos del Laboratorio

- Practicar técnicas legalmente
- Crear ambientes aislados
- Evitar impactos externos
- Realizar offensive security controlada

Configuraciones Propuestas

****Configuración 1****: Laptop/Desktop + sistemas públicos autorizados

- Mínimo: Solo máquina host
- Máximo: Acceso a plataformas CTF autorizadas

****Configuración 2****: Laptop/Desktop + Ubuntu VMs

- Máquina host + 2-3 VMs Ubuntu
- Ideal para principiantes

****Configuración 3****: Kali Linux + Ubuntu/Windows VMs

- Máquina Kali como atacante
- VMs objetivo como defensores

****Configuración 4****: Kali Linux + Vulnerable VMs

- Máquina Kali especializada
- VMs con vulnerabilidades conocidas (DVWA, Metasploitable)

Requisitos del Laboratorio Ideal

- ****Host puede acceder a las VMs****: Comunicación bidireccional
- ****VMs pueden verse entre sí****: Networking interno
- ****VMs NO deben acceder directamente al host****: Seguridad del host
- ****VMs deben tener Internet****: Acceso a recursos externos

Configuración de Networking

La configuración de networking es crítica para que herramientas de scanning funcionen correctamente.

VirtualBox: Usar **Bridged Adapter** para permitir comunicación entre VMs y acceso desde host.

Hyper-V: Configurar **Virtual Switch** con networking apropiado.

Requisitos de Hardware

La lección recomienda **mínimo 8 GB RAM** porque:

- VMs consumen CPU, RAM, almacenamiento y networking
- Con poca memoria, el host se vuelve lento
- Las VMs pueden colapsar
- El laboratorio pierde estabilidad

Modificación de Resolución en Hyper-V

Para cambiar resolución de pantalla de VMs en Hyper-V:

```
set-vmvideo "Ubuntu 20.04" -horizontalresolution:2048 -  
verticalresolution:1080 -resolutiontype single
```

8. Python Virtual Environment (venv)

Creación de venv

Linux/macOS:

```
python3 -m venv /ruta/mi_venv
```

Windows:

```
python3 -m venv C:\ruta\mi_venv
```

Activación de venv

Linux/macOS:

```
source venv/bin/activate
```

Windows:

```
venv\Scripts\activate.bat
```

Beneficios de venv

- **Aislamiento de dependencias**: Cada proyecto tiene sus propias librerías
- **Múltiples versiones**: Diferentes proyectos pueden usar versiones diferentes
- **Independencia del sistema**: No afecta Python global
- **Testing seguro**: Probar sin romper el sistema

Ejemplo de Aislamiento

```
# Sin venv - módulo disponible globalmente
python3 -c "import icmplib" # Funciona

# Con venv - módulo no disponible
source venv/bin/activate
python3 -c "import icmplib" # Error: ModuleNotFoundError

# Instalar en venv
pip install icmplib
python3 -c "import icmplib" # Funciona
```

9. Tecnologías Modernas de Laboratorio

Containerización

Tecnologías como Docker y Podman permiten:

- Ambientes reproducibles
- Laboratorios rápidos
- Despliegues efímeros
- Ambientes vulnerables reutilizables

Plataformas Vulnerables

- ****DVWA**** (Damn Vulnerable Web Application): Aplicación web vulnerable
- ****Metasploitable****: Sistema Linux vulnerable
- ****WebSploit****: Entorno de laboratorio web
- ****HackTheBox****: Plataforma de CTF

Ventajas de Containers

- Despliegue rápido
- Reproducibilidad
- Escalabilidad
- Aislamiento
- Portabilidad

10. Implicancias de Ciberseguridad

El laboratorio es fundamental para:

- ****Penetration Testing****: Practicar en ambientes controlados
- ****Red Teaming****: Simular ataques sin impacto
- ****Threat Hunting****: Investigar sin afectar producción
- ****Malware Analysis****: Analizar malware de manera segura
- ****DevSecOps****: Integrar seguridad en CI/CD
- ****Exploit Development****: Desarrollar exploits controladamente

11. Riesgos y Ética

Consideraciones Legales

****NUNCA practicar técnicas ofensivas:****

- Sobre terceros
- Sistemas públicos
- Sistemas sin autorización

Los laboratorios existen precisamente para:

- Reducir riesgos legales
- Evitar daños accidentales
- Practicar de forma ética

Implicancias Éticas

- ****Autorización****: Obtener autorización escrita
- ****Alcance****: Limitarse a sistemas autorizados
- ****Responsabilidad****: Reportar hallazgos responsablemente
- ****Confidencialidad****: Mantener información sensible segura

12. Buenas Prácticas

1. ****Laboratorio Aislado****: Separar completamente de producción
2. ****Snapshots****: Guardar estados para revertir rápidamente
3. ****Documentación****: Registrar configuración y cambios
4. ****Versionado****: Usar control de versiones para scripts
5. ****Automatización****: Automatizar despliegue de laboratorios
6. ****Seguridad****: Proteger credenciales y datos sensibles
7. ****Backup****: Mantener backups de configuraciones

13. Troubleshooting

Problema	Causa Probable	Solución
VM lenta	Recursos insuficientes	Aumentar RAM/CPU asignada
Sin conectividad	Networking mal configurado	Verificar Virtual Switch/Adapter
PermissionError	Privilegios insuficientes	Ejecutar con `sudo`
ModuleNotFoundError	Librería no instalada	Ejecutar `pip install [librería]`
Snapshot no funciona	Hypervisor no lo soporta	Verificar capacidades del hypervisor

14. Conclusiones

Esta lección marca la transición hacia ambientes profesionales de ciberseguridad. Los conceptos aprendidos son fundamentales para:

- **Red Team**: Operaciones ofensivas coordinadas
- **Blue Team**: Defensa y respuesta a incidentes
- **Malware Analysis**: Análisis seguro de malware
- **Digital Forensics**: Análisis forense de sistemas
- **DevSecOps**: Integración de seguridad en desarrollo
- **Offensive Security**: Penetration testing profesional
- **Threat Research**: Investigación de amenazas

Un profesional moderno de ciberseguridad debe ser capaz de:

- Crear laboratorios reproducibles
- Automatizar ambientes
- Aislar dependencias
- Construir entornos seguros
- Revertir estados rápidamente
- Documentar configuraciones
- Practicar de forma ética y legal

Los scripts de esta lección proporcionan herramientas de diagnóstico para validar que el laboratorio está correctamente configurado y operativo. En lecciones posteriores, estos laboratorios servirán como base para técnicas más avanzadas de ciberseguridad.